ecma INTERNATIONAL

Draft Standard TEA

1.0 (draft) / May 17, 2026

Transparency Exchange API specification

Standard

Rue du Rhône 114 CH-1204 Geneva T: +41 22 849 6000 F: +41 22 849 6001

Ecma International
Rue du Rhone 114 CH-1204 Geneva
Tel: +41 22 849 6000
Fax: +41 22 849 6001
Web: https://www.ecma-international.org

is the registered trademark of Ecma International

Contents

page

1 Scope . . . 1
2 Conformance . . . 1
3 Normative references . . . 1
4 Terms and definitions . . . 1
5 Specification narrative . . . 2
5.1 TEA Requirements . . . 2
5.2 TEA Use Cases . . . 4
5.3 Transparency Exchange API - Discovery . . . 6
5.4 Transparency Exchange API - Authentication and authorization . . . 11
5.5 Transparency Exchange API: Consumer access . . . 13
5.6 Overview of the TEA API from a producer standpoint . . . 16
5.7 The TEA product API . . . 17
5.8 TEA Product Release . . . 19
5.9 The TEA Component API . . . 20
5.10 TEA Component Release . . . 22
5.11 TEA Releases and Collections . . . 23
5.12 Known types . . . 31
5.13 Transparency Exchange API - Trusting digital signatures in TEA . . . 33
6 The TEA API . . . 35
6.1 CLE . . . 35
6.2 TEA Artifact . . . 37
6.3 TEA Component . . . 38
6.4 TEA Component Release . . . 39
6.5 TEA Discovery . . . 41
6.6 TEA Product . . . 42
6.7 TEA Product Release . . . 43
7 Data model . . . 45
7.1 artifact . . . 45
7.2 artifact-format . . . 47
7.3 artifact-type . . . 47
7.4 checksum . . . 48
7.5 checksum-type . . . 48
7.6 cle . . . 49
7.7 cle-definitions . . . 50
7.8 cle-event . . . 51
7.9 cle-event-type . . . 52
7.10 cle-support-definition . . . 52
7.11 cle-version-specifier . . . 53
7.12 collection . . . 53
7.13 collection-belongs-to-type . . . 56
7.14 collection-update-reason . . . 56
7.15 collection-update-reason-type . . . 56
7.16 compliance-document-type . . . 57
7.17 component . . . 57
7.18 component-ref . . . 58
7.19 component-release-with-collection . . . 59
7.20 date-time . . . 61
7.21 discovery-info . . . 61
7.22 error-response . . . 61
7.23 identifier . . . 61
7.24 identifier-type . . . 62
7.25 paginated-component-release-response . . . 62
7.26 paginated-component-response . . . 62
7.27 paginated-product-release-response . . . 62
7.28 paginated-product-response . . . 62
7.29 pagination-details . . . 62

7.30 product **63**
7.31 productRelease **63**
7.32 release **64**
7.33 release-distribution **67**
7.34 tea-server-info **69**
7.35 unknown-error-type **70**
7.36 uuid **70**
Annex A (informative) **Bibliography** **71**
Annex B (informative) **Colophon** **73**
Software License **75**

Introduction

The TEA API is created to support automation of the software supply chain. Upstream vendors and open source projects can use this standard to keep downstream consumers up to date with transparency artefacts such as, but not limited to, bill of materials, VEX files, attestations and much more.

This specification defines a standard, format agnostic, API for the exchange of product related artefacts, like BOMs, between systems. The work includes:

- Discovery of servers: Describes discovery using the Transparency Exchange Identifier (TEI)
- Retrieval of artefacts
- Publication of artefacts
- Authentication and authorization
- Querying

System and tooling implementors are encouraged to adopt this API standard for sending/receiving transparency artefacts between systems. This will enable more widespread "out of the box" integration support in the BOM ecosystem.

About this specification

This document is an experimental ECMA-style rendering of the OWASP Transparency Exchange API (TEA), produced as a proof of concept for ECMA TC54 TG1 publication tooling.

This specification is developed on GitHub:

Upstream repository: https://github.com/CycloneDX/transparency-exchange-api
Experimental ECMA rendering: https://github.com/relizaio/transparency-exchange-api

Transparency Exchange API specification

1 Scope

SKELETON — to be authored by the working group.

This Specification defines the Transparency Exchange API (TEA), a format-agnostic protocol for the exchange of software, hardware and system transparency artefacts — including, but not limited to, Bills of Materials (xBOMs), attestations, vulnerability disclosure documents, and lifecycle events — between publishers and consumers.

The Specification covers:

- The Transparency Exchange Identifier (TEI) and its resolution.
- The data model used to describe products, components, releases, collections and artefacts.
- The HTTP API surface used to discover, retrieve and (in future revisions) publish transparency artefacts.
- Authentication and authorization patterns expected of conformant implementations.

The Specification does *not* define the internal format of the transparency artefacts themselves; those are defined by their respective standards (e.g. CycloneDX, SPDX, OpenVEX, CSAF, CDXA).

2 Conformance

SKELETON — to be authored by the working group.

A conformant TEA implementation shall:

- Implement the consumer API surface defined in 6 in full.
- Resolve Transparency Exchange Identifiers per the Discovery chapter.
- Honour the authentication and authorization model defined in the Authentication chapter.
- Use the data model defined in 7 for request and response payloads.

The terms **shall**, **shall not**, **should**, **should not**, and **may** are to be interpreted as described in IETF RFC 2119 and RFC 8174.

3 Normative references

SKELETON — to be authored by the working group.

The following referenced documents are indispensable for the application of this Specification.

- IETF RFC 2119 — Key words for use in RFCs to Indicate Requirement Levels.
- IETF RFC 8174 — Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words.
- IETF RFC 8141 — Uniform Resource Names (URNs).
- IETF RFC 7519 — JSON Web Token (JWT).
- IETF RFC 9457 — Problem Details for HTTP APIs.
- OpenAPI Specification, version 3.1.1.
- ECMA-424 — CycloneDX Bill of Materials Specification.
- ECMA-428 — Common Lifecycle Enumeration (CLE).

4 Terms and definitions

SKELETON — to be authored by the working group.

For the purposes of this Specification, the following terms and definitions apply.

API
Application Programming Interface.

Authentication (authn)
The process of verifying the identity of a principal.

Authorization (authz)
The process of determining what a verified principal is permitted to do.

Collection
A versioned set of TEA Artifacts associated with a specific TEA Release.

Product
An item sold, distributed or otherwise delivered under one name.

Product Release
A specific version of a Product that can be acquired or downloaded.

TEA Artifact
A named file (e.g. an SBOM, an attestation, a VEX document) referenced by a Collection.

TEI
Transparency Exchange Identifier — a URN that resolves to a TEA Product Release.

5 Specification narrative

The following sections are derived from the working group's authoring notes maintained as Markdown in the source repository.

5.1 TEA Requirements

5.1.1 Repository discovery

Based on an identifier a repository URL needs to be found. The identifier can be:

- PURL
- Product name or Product SKU and vendor name
- EAN bar code
- Product SKU
- Vendor UUID
- Hash of object

At the base URL well known URLs (ref) needs to point to

- A lifecycle status document (using OWASP Common Lifecycle Enumeration, CLE)
- A version list. For each version, a URL will point to where a **collection** can be found
- Vendor Discovery, returns a list of Vendors represented in the repository
  - Vendor Name
  - Vendor ID

As an alternative, discovery using a company's ordinary web site should be supported. This can be handled using the file security.txt (IETF RFC 9116)

5.1.2 Artefact Discovery based on TEA collections

The API MUST provide a way to discover the artefacts that are available for retrieval or further query. Discovery SHOULD group artefacts together that represent a **collection** that are directly applicable to a given product with a given version. Collections are OPTIONAL.

- SBOM - Software Bill of Material
- CBOM - Cryptography Bill of Material
- HBOM - Hardware Bill of Material
- VDR - Vulnerability Disclosure Report
- VEX - Vulnerability Exploitability eXchange
- CDXA - Attestation

Authn/Authz MUST be supported

5.1.3 Collection Management

The API SHOULD provide a method to manage collections, such as adding new collections, modifying collections, or deleting existing collections.

- Authn/Authz MUST be supported

5.1.4 Artefact Retrieval

The API MUST provide a method in which to retrieve an artefact based on the identity of the artefact. For example, using CycloneDX BOM-Link to retrieve either the latest version or specific version of an artefact.

```
urn:cdx:serialNumber
urn:cdx:serialNumber/version
```

The API needs to provide support for update checks, i.e. to check if a document is updated without downloading. (possibly etag or HEAD method or similar) Authn/Authz MUST be supported

5.1.5 Artefact Publishing

The API MUST provide a way to publish an artefact, either standalone or to a collection. The detection of duplicate artefacts with the same identity MUST be handled and prevented. Authn/Authz MUST be supported

5.1.6 Artefact Versioning

The system and API must support artefact versioning for formats that support versioning such as CycloneDX. For example:

- The ability to retrieve the latest SBOM vs a previous (uncorrected) version of the same SBOM. Corrections to SBOMs is a supported use case in the NTIA framing document.
- The ability to retrieve the latest VEX along with previous VEX for the same product so that time-series decisions are transparently available.

Authn/Authz MUST be supported

5.1.7 insights: Search Artefact Inventory

The API MUST provide a way to search the inventory of a specific BOM or all available BOMs for a given component or service. The API SHOULD support multiple identity formats including PURL, CPE, SWID, GAV, GTIN, and GMN.

For example:

- Return the identity of all BOMs that have a vulnerable version of Apache Log4J: **`pkg:maven/org.apache.logging.log4j/log4j-core@2.10.0`**

The API MUST provide a way to search for the metadata component across all available BOMs. The API SHOULD support multiple identity formats including PURL, CPE, SWID, GAV, GTIN, and GMN. For example:

- Return the identity of all artefacts that describe **`cpe:/a:acme:commerce_suite:1.0`**.

Authn/Authz MUST be supported

5.2 TEA Use Cases

An overview of use cases to consider for the Transparency Exchange API. The use cases are marked with a short code for reference. All use cases needs a story like "Alice has bought a..."

The use cases are divided in two categories:

- Use cases for **customers** (end-users, manufacturers) to find a repository with Transparency artefacts for a single unit purchased
- Use cases where there are different **products**
  - This applies after discovery where we need to handle various things a customer may buy as a single unit

5.2.1 Customer focused use cases

5.2.1.1 C1: Consumer: Automated discovery based on SBOM identifier

As a consumer that has an SBOM for a product, I want to be able to retrieve VEX and VDR files automatically both for current and old versions of the software. In the SBOM the product is identified by a PURL or other means (CPE, ...)

5.2.1.2 C2: Consumer: Automation based on product name/identifier

As a consumer, I want to download artefacts for a product based on known data. A combination of manufacturer, product name, vendor product ID, EAN bar code or other unique identifier. After discovering the base repository URL I want to be able to find a specific product variant and version.

If the consumer is a business, then the procurement process may include delivery of an SBOM with proper identifiers and possibly URLs or identifiers in another document, which may bootstrap the discovery process in a more exact way than in the case of buying a product in a retail market. Alice bought a gadget at the gadget store that contains a full Linux system. Where and how will she find the SBOM and VEX for the gadget?

5.2.1.3 C3: Consumer: Artefact retrieval

As a consumer, I want to retrieve one or more supply chain artefacts for the products that I have access to, possibly through licensing or other means. As a consumer, I should be able to retrieve all source artefacts such as xBOMs, VDR/VEX, CDXA, and CLE.

5.2.1.4 C4: Consumer: Summarized CLE

As a consumer, I want the ability to get the current lifecycle values for a given product. A CLE captures all lifecycle events over time, however, there is a need to retrieve only the current values for things like product name, vendor name, and milestone events.

5.2.1.5 C5: Consumer: Insights

As a consumer, I want the ability to simply ask the API questions rather than having to download, process, and analyze raw supply chain artefacts on my own systems. Common questions should be provided by the API by default along with the ability to query for more complex answers using the Common Expression Language (CEL).

NOTE: Project Hyades (Dependency-Track v5) already implements CEL with the CycloneDX object model and has proven that this approach works for complex queries.

5.2.1.6 D1: Developer/PM - Components for inclusion in products by other projects/ developers

A developer needs a component - software, library (open source and/or proprietary) to include in a product - publicly available (may be commercial) or in an internal system. Developer can be individual, company or public sector. They need insight into the library or component and be able to automatically keep upstream artefacts up to date.

5.2.1.7 B1: Business consumer

Acme LLC buys 3 `000` gadgetrons from Emca LTD to be distributed over a retail chain. Acme runs an in-house vulnerability management system (Dependency Track) to manage SBOMs and download VEX files to check for vulnerabilities. Acme has products from exactly 14.385 vendors in the system. How will their systems get continuous access to current and old documents - attestations, SBOM, VEX and other files?

5.2.1.8 E1: Third party: Regulators

Alice & Bob Enterprises AB has gotten a EUCC certification to get their Whola Firewall certified for CRA-compatible CE labeling. In order to maintain the certification the certifying body needs access to SBOM and VEX updates from A&BE in an automated way.

5.2.1.9 E2: External potential customer - insights before purchase

Palme Auditors INC wants to buy the ACME SWISH product from a vendor. They want to examine vulnerability handling and get some insights into the products before making a decision.

5.2.1.10 E3: Hacker or competition

There are non-customers and not-to-be-customers that wants to get insight into how a product is composed by getting the transparency docs.

5.2.1.11 E4: Open Source user

Palme Inc considers using the asterisk.org <http://asterisk.org> open source telephony PBX. They need to make an assessment before starting tests and possible production use. The can either use the Debian package, the Alpine Linux Package or build a binary themselves from source code. How can they find the transparency exchange data sets?

5.2.1.12 O1: Open Source project

The hatturl open source project publish a library and a server side software on Github. This is later packaged as packages in many Linux distributions.

How does the project publish artefacts? What are the requirements?

5.2.2 Product focused use cases

5.2.2.1 P1: A standalone app

Customer buys a standalone application that is delivered as a binary

5.2.2.2 P2: A OCI container

Customer gets a container with many third party components and a binary software developed by the manufacturer

5.2.2.3 P3: Embedded system

5.2.2.4 P4: Package with many devices

5.2.2.5 P5: An Open Source software library

5.3 Transparency Exchange API - Discovery

5.3.1 From product identifier to API endpoint

TEA Discovery is the connection between a product release identifier and the API endpoint. A "product release" is something that the customer aquires or downloads - hardware and/or software.

It can be a bundle of many digital devices or software applications. A "product release" normally also has an entry in a large corporation's asset inventory system.

A product release identifier is embedded in a URN where the identifier is one of many existing identifiers or a random string - like an EAN or UPC bar code, UUID, product number or PURL.

The goal is for a user to add this URN to the transparency platform (sometimes with an associated authentication token) and have the platform access the required artefacts in a highly automated fashion.

5.3.2 Advertising the TEI

The TEI for a product release can be communicated to the user in many ways.

- A QR code on a box
- On the invoice or delivery note
- For software with a GUI, in an "about" box

The user needs to get the TEI from the manufacturer, through a reseller or directly. The TEI is defined by the manufacturer and can normally not be derived from known information.

5.3.3 TEA Discovery - defining an extensible identifier

TEA discovery is the process where a user with a product release identifier can discover and download artefacts automatically, with or without authentication. A globally unique identifier is required for a given product release. This identifier is called the Transparency Exchange Identifier (TEI).

The TEI identifier is based on DNS, which assures a uniqueness per vendor (or open source project) and gives the vendor a namespace to define product release identifiers based on existing or new identifiers like EAN/UPC bar code, PURLs or other existing schemes. A given product release may have multiple identifiers as long as they all resolve into the same destination.

The vendor needs to make sure that the TEI is unique within the vendor's namespace. There is no intention to create any TEI registries.

5.3.4 The TEI URN: An extensible identifier

The TEI, **Transparency Exchange Identifier**, is a URN schema that is extensible based on existing identifiers like EAN codes, PURL and other identifiers. It is based on a DNS name, which leads to global uniqueness without new registries.

The TEI can be shown in the software itself, in shipping documentation, in web pages and app stores.

A TEI belongs to a single product release. A product release can have multiple TEIs - like one with a EAN/UPC barcode and one with the vendor's product number.

5.3.4.1 TEI syntax

The TEI consists of three core parts

```
urn:tei:<type>:<domain-name>:<unique-identifier>
```

- The **`type`** which defines the syntax of the unique identifier part
- The **`domain-name`** part resolves into a web server, which may not be the API host.
  - The uniqueness of the name is the domain name part that has to be registred at creation of the TEI.
- The **`unique-identifier`** has to be unique within the **`domain-name`**. Recommendation is to use a UUID but it can be an existing article code too

Note: this requires a registration of the TEI URN schema with IANA - see here <https://github.com/CycloneDX/transparency-exchange-api/issues/18>

5.3.4.2 TEI types

The below show examples of TEI where the types are specific known formats or types.

Reminder: the **`unique-identifer`** component of the TEI needs only be unique within the **`domain-name`**.

5.3.4.2.1 PURL - Package URL

Where the **`unique-identifier`** is a PURL in it's canonical string form.

Syntax:

```
urn:tei:purl:<domain-name>:<purl>
```

Example:

```
urn:tei:purl:cyclonedx.org:pkg:pypi/cyclonedx-python-lib@8.4.0?extension=whl&qualifi
```

5.3.4.2.2 SWID

Where the **unique-identifier** is a SWID.

Syntax:

```
urn:tei:swid:<domain-name>:<swid>
```

Note that there is a TEI SWID type as well as a PURL SWID type.

5.3.4.2.3 HASH

Where the **unique-identifier** is a Hash. Supports the following hash types:

- SHA256
- SHA384
- SHA512

```
urn:tei:hash:<domain-name>:<hashtype>:<hash>
```

Example:

```
urn:tei:hash:cyclonedx.org:SHA256:fd44efd601f651c8865acf0dfeacb0df19a2b50ec69ead0262
```

The origin of the hash is up to the vendor to define.

5.3.4.2.4 UUID

Where the **unique-identifier** is a UUID.

Syntax:

```
urn:tei:uuid:<domain-name>:<uuid>
```

Example:

```
urn:tei:uuid:cyclonedx.org:d4d9f54a-abcf-11ee-ac79-1a52914d44b1
```

5.3.4.2.5 EAN/UPC

Where the **unique-identifier** is a EAN/UPC.

Syntax:

```
urn:tei:eanupc:<domain-name>:<ean/upc-number>
```

Example:

```
urn:tei:eanupc:cyclonedx.org:1234567890123
```

5.3.4.2.6 GTIN

Where the **unique-identifier** is a GTIN <https://www.gs1.org/standards/id-keys/gtin>.

Syntax:

```
urn:tei:gtin:<domain-name>:<gtin-number>
```

Example:

`urn:tei:gtin:cyclonedx.org:0234567890123`

5.3.4.2.7 ASIN

Where the **`unique-identifier`** is a ASIN <https://sell.amazon.com/blog/what-is-an-asin>.

Syntax:

`urn:tei:asin:<domain-name>:<asin-identifier>`

Example:

`urn:tei:asin:cyclonedx.org:B07FZ8S74R`

5.3.4.2.8 UDI

Where the **`unique-identifier`** is a UDI <https://www.gs1.org/industries/healthcare/udi>.

Syntax:

`urn:tei:udi:<domain-name>:<udi-identifier>`

Example:

`urn:tei:udi:cyclonedx.org:00123456789012`

Note that if the same identifier, like EAN, is used for multiple different product releases then this EAN code will not be unique for a given product. While this case is supported by TEA, the vendor is recommended to create a separate TEI for each unique product sold, like UUID or hash. In any case, the vendor SHOULD minimize the number of distinct product releases returned per TEI. Preferable situation is to have a single product release per TEI.

5.3.4.3 TEI resolution using DNS

The **`domain-name`** part of the TEI is used in a DNS query to find one or multiple locations for product transparency exchange information.

At the URL a well-known name space is used to find out where the API endpoint is hosted. This is solved by using the ".well-known" name space as defined by the IETF.

- **`urn:tei:uuid:products.example.com:d4d9f54a-abcf-11ee-ac79-1a52914d44b1`**
- Syntax: **`urn:tei:uuid:<name based on domain>:<unique identifier>`**

The name in the DNS name part points to a set of DNS records.

A TEI with **`domain-name tea.example.com`** queries DNS for **`tea.example.com`**, considering **`A`**, **`AAAA`** and **`CNAME`** records. These point to the hosts available for the Transparency Exchange API.

The TEA client connects to the host using HTTPS and validates the certificate. The URL is composed of the host name with the **`/.well-known/tea`** path added.

This results in the base URL such as **`https://products.example.com/.well-known/tea`**

This response must contain json object that lists the available TEA server endpoints and supported versions. The json must conform to the TEA Well-Known Schema.

Example:

```
{
  "schemaVersion": 1,
  "endpoints": [
    {
      "url": "https://api.teaexample.com",
      "versions":
        [
          "0.1.0-beta.1",
          "0.2.0-beta.2",
          "1.0.0"
        ],
      "priority": 1
    },
    {
      "url": "https://api2.teaexample.com/mytea",
      "versions":
        [
          "1.0.0"
        ],
      "priority": 0.5
    }
  ]
}
```

5.3.5 Port resolution

Currently, the port number is not part of the TEI but it is needed to connect to the API. A port number cannot be added to the TEI URN spec as it breaks the location independence requirement of URN.

The TEA API server may be hosted on any port, but the server that is part of the first step of discovery will by default be running on the default HTTPS port 443. In the JSON file found there, the server URLs may contain port numbers.

If the clients are known to support HTTPS/SVCB DNS records for failover and load balancing, these may be used to redirect to other ports and servers.

Public servers are recommended to have the server hosting the DNS name used in the TEI URI running on the default port to enable discovery of the API servers.

5.3.6 Connecting to the API

Clients must pick any one of the endpoints listed in the **`.well-known/tea`** json response. The client MUST pick an endpoint with the at least one version that is supported by the client is using. The client MUST prioritize endpoints with the highest matching version supported both by the client and the endpoint based on SemVer 2.0.0 specification comparison rules <https://semver.org/#spec-item-11>. If there are several endpoints like these and if the priority field is present, the client SHOULD pick the endpoint with the highest priority value (a float between 0 and 1).

The client must then construct the full URL to the API by appending the "/v" plus one of the versions listed in the **`versions`** array of the selected endpoint, plus "/discovery?tei=", plus the TEI that is url-encoded according to [RFC3986] and [RFC3986]).

Examples:

1. For TEI **`urn:tei:uuid:products.example.com:d4d9f54a-abcf-11ee-ac79-1a52914d44b`**
   `https://api.teaexample.com/v0.2.0-beta.2/discovery?tei=urn%3Atei%3Auuid%3Aproduct`
2. For

TEI
urn:tei:purl:products.example.com:pkg:deb/debian/curl@7.50.3-1?arch=i386&distro=j
https://api2.teaexample.com/mytea/v1.0.0/discovery?tei=urn%3Atei%3Apurl%3Aproduct

The discovery endpoint is a part of the TEA OpenAPI specification.

If the TEI is known to the TEA server, the discovery endpoint must return at least the product release uuid, the root URL of the TEA server, the list of supported versions, plus the response may have other fields based on the current version of the TEA OpenAPI specification.

If the TEI is not known to the TEA server, the discovery endpoint must return a 404 status code with a response describing the error.

If the DNS record for the discovery endpoint cannot be resolved by the client, or the discovery endpoint fails with 5xx error code, or the TLS certificate cannot be validated, the client MUST retry the discovery endpoint with the next endpoint in the list, if another endpoint is present. While doing so the client SHOULD preserve the priority order if provided (from highest to lowest priority). If no other endpoint is available, the client MUST retry the discovery endpoint with the first endpoint in the list. The client SHOULD implement an exponential backoff strategy for retries.

Client implementations needs to indicate authentication errors clearly to the users, to indicate that there are no updates. An expired token or TLS Client Cert will mean that new versions of a product or updated artefacts will not be accessed. Authentication error codes (401, 403) should not lead to failover to the next endpoint in the list.

How this is communicated to the client users is implementation specific.

5.3.7 Notes Regarding .well-known

Servers MUST NOT locate the actual TEA service endpoint at the **`.well-known`** URI as per Section 1.1 of [RFC5785].

5.3.7.1 TLS Encryption

The .well-known endpoint must only be available via HTTPS. Using unencrypted HTTP is not valid.

- TEI: **`urn:tei:uuid:products.example.com:d4d9f54a-abcf-11ee-ac79-1a52914d44b1`**
- URL: **`https://products.example.com/.well-known/tea`**

5.3.8 References

- IANA .well-known registry <https://www.iana.org/assignments/well-known-uris/well-known-uris.xhtml>
- IANA URI registry <https://www.iana.org/assignments/urn-namespaces/urn-namespaces.xhtml#urn-namespaces-1>

5.4 Transparency Exchange API - Authentication and authorization

This document covers authentication and authorization on the consumer side of a TEA service - the discovery and download of software transparency artefacts.

5.4.1 Requirements

Authorization: A user of a TEA service may get access to all objects (components, collections) and artefacts or just a subset, depending on the publisher of the data. Authorization is connected to **authentication**.

The level of authorization is up to the implementer of the TEA implementation and the publisher, whether an identity gets access to all objects in a service or just a subset.

In order to get interoperability between clients and servers implementing the protocol, the specification focuses on the authentication. After successful authentication, the authorization may be implemented in multiple ways - on various levels of the API - depending on what information the user can access.

As an example, one implementation may publish all information about existing artefacts and software versions openly, but restrict access to artefacts to those that match the customers installation. Another implementation can implement a filter that does not show products and versions ("components") that the customer has not aquired.

For most Open Source projects, implementing authentication - setting up accounts and managing authorization - does not make much sense, since the information is usually in the open any way.

This specification does not impose any requirement on authentication on a TEA service. But should the provider implement authentication, two methods are supported in order to promote interoperability.

- HTTP Bearer Token Authentication
- Mutual TLS with verifiable client and server certificates

A client may use both HTTP bearer token auth and TLS client certificates when accessing multiple TEA services. It is up to the service provider to select authentication.

5.4.2 HTTP bearer token auth

The API will support HTTP bearer token in the **Authorization:** http header. How the token is aquired is out of scope for this specification, as is the potential content of the token.

As an example the token can be downloaded from a customer support portal with a long-term validity. This token is then installed into the software transparency platform (the TEA client) and used to automatically retrieve software transparency artefacts.

For each TEA service, one bearer token is needed. The token itself (or the backend) should specify authorization for the token.

5.4.3 Mutual TLS

For Mutual TLS the client certificates will be managed by the service and provided in a service-specific way. Clients should be able to configure a separate client certificate (and private key) on a per-service level, not assuming that a client certificate for one service is trusted anywhere else.

5.4.4 References

- RFC 6750: The Oauth 2.0 Authorization Framework: Bearer Token Usage (https://www.rfc-editor.org/rfc/rfc6750)

5.5 Transparency Exchange API: Consumer access

The consumer access starts with a TEI, A transparency Exchange Identifier. This is used to find the API server as described in the discovery document.

5.5.1 API usage

The standard TEI points to a product release. A product release is something sold, downloaded as an opensource project or aquired by other means. It contains one or multiple component releases.

- **List of TEA Component Rleases**: Component releases are components of a product release. Each Component release has its own versioning and its own set of artefacts, they have a timestamp and a lifecycle enumeration. They are normally sorted by timestamps. The TEA API has no requirements of type of version string (semantic or any other scheme) - it's just an identifier set by the manufacturer.
- **List of TEA Collections**: For each release, there is a list of TEA collections as indicated by release date and a version integer starting with collection version 1.
- **List of TEA Artifacts**: The collection is unique for a version and contains a list of artefacts. This can be SBOM files, VEX, SCITT, IN-TOTO or other documents. Note that a single artefact can belong to multiple Component or Product Releases.
- **List of artefact formats**: An artefact can be published in multiple formats.

The user has to know product release TEI and in some cases version of each component (TEA Component Release) to find the list of artefacts for the particular Product Release.

5.5.2 API flow based on TEI discovery

```
---
title: TEA consumer flow
---
sequenceDiagram
    autonumber
    actor manufacturer as Manufacturer
    actor user as TEA Client

    participant discovery as TEA Discovery / TEA Server
    participant tea_product_release as TEA Product Release
    participant tea_component_release as TEA Component Release

    manufacturer ->> user: Provides TEI (Transparency Exchange Identifier)

    user ->> discovery: GET https://<domain>/.well-known/tea
    discovery -->> user: TEA discovery document (API servers & endpoints)

    user ->> discovery: Call Discovery endpoint with TEI
    discovery -->> user: Product Release reference(s)

    alt Multiple Product Releases returned
        user ->> tea_product_release: Resolve Product Release(s)
        tea_product_release -->> user: List of Component Releases
        user ->> user: Identify Discriminating Component Releases
        user ->> tea_component_release: Resolve Discriminating Component Releases
        tea_component_release -->> user: Discriminating Component Release Details
        user ->> user: Converge on a Single Product Release
    end

    user ->> tea_product_release: Resolve Product Release Details
    tea_product_release -->> user: List of Component Releases

    loop For each tea_component_release
        user ->> tea_component_release: Obtain latest collections
        tea_component_release -->> user: List of TEA Artifacts
    end
```

5.5.3 API flow based on direct access to API

In this case, the client wants to search for a specific product release using the API

```
---
title: TEA client flow with search
---

sequenceDiagram
    autonumber
    actor user

    participant tea_product_release as TEA Product Release
    participant tea_component_release as TEA Component Release
    participant tea_collection as TEA Collection
    participant tea_artifact as TEA Artifact


    user ->> tea_product_release: Search for product releases based on identifier ((
    tea_product_release ->> user: List of product releases

    user ->> tea_product_release: Finding all product parts (TEA Component Releases)
    tea_product_release ->> user: List of TEA Component Releases

    user ->> tea_component_release: Finding information of a component release
    tea_component_release ->> user: List of releases and collection id for each rele

    user ->> tea_collection: Finding all TEA Artifacts for TEA Component Release
    tea_collection ->> user: List of TEA Artifacts and formats available for each TE

    user ->> tea_artifact: Request to download TEA artifact
    tea_artifact ->> user: TEA Artifact
```

5.5.4 API flow based on cached data - checking for a new release

In this case a TEA client knows the component UUID and wants to check the status of the used release and if there's a new release. The client may limit the query with a given date for a release.

```
---
title: TEA client flow with direct query for release
---

sequenceDiagram
    autonumber
    actor user

    participant tea_component as TEA Component
    participant tea_component_release as TEA Component Release

    user ->> tea_component: Finding a specific version/release
    tea_component ->> user: List of releases and collection id for each release

    user ->> tea_component_release: Details for discovered release
    tea_component_release ->> user: Collection with artefact details for the release
```

5.5.5 API flow based on cached data - checking if a collection changed

In this case a TEA client knows the release UUID, the collection UUID, and the collection version from previous queries. If the given version is not the same, another query is done to get reason for update and new collection list of artefacts.

```
---
title: TEA client collection query
---

sequenceDiagram
    autonumber
    actor user

    participant tea_collection as TEA Collection


    user ->> tea_collection: Finding the current collection, including version
    tea_collection ->> user: List of artefacts and formats available for each artefa

    user ->> tea_collection: Request to access previous version of the collection to
    tea_collection ->> user: Previous version of collection
```

5.6 Overview of the TEA API from a producer standpoint

This is input for the working group.

5.6.1 Bootstrapping

```
sequenceDiagram
    autonumber
    actor Vendor
    participant tea_product as TEA Product
    participant tea_component as TEA Component
    participant tea_collection as TEA Collection

    Vendor ->> tea_product: POST to /v1/product to create new product
    tea_product -->> Vendor: Product is created and TEA Product Identifier (PI) retu

    Vendor ->> tea_component: POST to /v1/component with the TEA PI and component ve
    tea_component ->> Vendor: Component is created and a TEA Component ID is returne

    Vendor ->> tea_collection: POST to /v1/collection with the TEA Component ID and
    tea_collection ->> Vendor: Collection is created with the collection ID returnec
```

5.6.2 Release life cycle

```
sequenceDiagram
    autonumber
    actor Vendor
    participant tea_product as TEA Product
    participant tea_component as TEA Component
    participant tea_collection as TEA Collection

    Note over Vendor,tea_component: Create new release

    Vendor ->> tea_component: POST to /v1/component with the TEA PI and component ve
    tea_component ->> Vendor: Component is created and a TEA Component ID is returne

    Note over Vendor,TEA Component: Add an artefact (e.g. SBOM)
    Vendor ->> tea_collection: POST to /v1/collection with the TEA Component ID and
    tea_collection ->> Vendor: Collection is created with the collection ID returnec
```

5.6.3 Adding a new artefact

```
sequenceDiagram
    autonumber
    actor Vendor
    participant tea_product as TEA Product
    participant tea_component as TEA Component
    participant tea_collection as TEA Collection

    Vendor ->> tea_component: GET to /v1/component with the TEA PI to get the latest
    tea_component ->> Vendor: Component will be returned

    Vendor ->> tea_collection: POST to /v1/collection with the TEA Component ID and
    tea_collection ->> Vendor: Collection is created with the collection ID returnec
```

5.7 The TEA product API

After TEA discovery, the Transparency Exchange Identifier (TEI) resolves to a specific TEA Product Release, which represents a concrete, versioned offering. A TEA Product is an optional higher-level object that groups multiple Product Releases for a product line or family and can be browsed via **`/product/{uuid}/releases`**.

- A product release may consist of a single component, the output will be metadata about the product and the TEA COMPONENT object.
- For a composed product release consisting of a bundle of components or component releases, the response will be multiple TEA COMPONENT objects.

In addition, all known TEIs for the product will be returned, in order for a TEA client to avoid duplication. This list can also include known Package URLs (PURL) and CPEs for the product.

5.7.1 Authorization

Authorization can be done on multiple levels, including which products and versions are supported for a specific user.

5.7.2 Composite products

A TEA Product Release will be the starting point of discovery. The TEA product release will list all included components

with the UUID of the TEA component. The reference list may also include a UUID of a specific release of a component in the case where a product always includes a single release of the component.

The URL can be to a different vendor or different site with the same vendor.

5.7.3 TEA Product object

A TEA Product object has the following parts:

- **uuid**: A unique identifier for the TEA product
- **name**: Product name
- **identifiers**: List of identifiers for the product
  - **idType**: Type of identifier, e.g. **`tei`**, **`purl`**, **`cpe`**
  - **idValue**: Identifier value
- **components**: List of TEA components for the product
  - **uuid**: Unique identifier of the TEA component
  - **release**: Optional UUID of a TEA component release

The TEA Component UUID is used in the Component API to find out which versions of the Component that exists.

The goal of the TEA Product API is to provide a selection of product versions to assist the user software in finding a match for the owned version.

5.7.3.1 Example

An example of a product consisting of an OSS project and all its Maven artefacts:


```
{
  "uuid": "09e8c73b-ac45-4475-acac-33e6a7314e6d",
  "name": "Apache Log4j 2",
  "identifiers": [
    {
      "idType": "CPE",
      "idValue": "cpe:2.3:a:apache:log4j"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-api"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-core"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-layout-template-json"
    }
  ]
}
```


Releases for this Product can be browsed via the API endpoint **`/product/{uuid}/releases`**.

5.7.3.2 API usage

The user will find this API end point using TEA discovery.

A user will approach the API just to discover data before purchase, or with a specific product and product version in scope. The format of the version may follow many syntaxes, so maybe the API needs to be able to provide some sort of format for the version string.

An automated system may want to provide the user with a GUI, listing versions and being able to scroll to the next page until the user selects a version.

5.7.3.3 Tea API operation

- Recommendation: Support HTTP compression
- Recommendation: HTTP content negotiation
  - Like "I prefer JSON, but can accept XML"
- Pagination support
  - max per page
  - start page
  - Default value defined

5.8 TEA Product Release

5.8.1 Overview

A TEA Product Release represents a specific versioned release of a TEA Product. It is the primary resolvable entity via TEI and the entry point for discovery of included components and related collections of security artefacts.

Key attributes:

- uuid: Unique identifier of the product release
- product: UUID of the TEA Product this release belongs to
- version: Human-readable version string of the product release
- createdDate: Timestamp when the product release was created in TEA
- releaseDate: Upstream product release timestamp
- preRelease: Indicates pre-release/beta status
- identifiers: Array of identifiers (idType: CPE/TEI/PURL; idValue: string)
- components: Array of component references included in this product release
  - uuid: UUID of the TEA Component
  - release: Optional UUID of a specific component release to pin an exact version

Collections for a product release contain artefacts relevant to that product release.

5.8.2 JSON examples

The following example is reused from the OpenAPI schema (**components/schemas/productRelease.examples**), ensuring exact field names and casing.


```
{
  "uuid": "123e4567-e89b-12d3-a456-426614174000",
  "version": "2.24.3",
  "createdDate": "2025-04-01T15:43:00Z",
  "releaseDate": "2025-04-01T15:43:00Z",
  "identifiers": [
    {
      "idType": "TEI",
      "idValue": "tei:vendor:product@2.24.3"
    }
  ],
  "components": [
    {
      "uuid": "3910e0fd-aff4-48d6-b75f-8bf6b84687f0"
    },
    {
      "uuid": "b844c9bd-55d6-478c-af59-954a932b6ad3",
      "release": "da89e38e-95e7-44ca-aa7d-f3b6b34c7fab"
    }
  ]
}
```


Notes:

- Property **product** exists in the schema and links a product release to its parent product; it may not be present in all examples.
- Use uppercase idType values exactly as defined by the schema enum: CPE, TEI, PURL.

5.9 The TEA Component API

The TEA Component represents a component lineage. A product release may be constructed with one or multiple TEA Components, each with their own set of releases and related artefacts.

Each TEA Component has a list of Component Releases (see **/component/{uuid}/releases**), which enumerates all known versions for that component.

The API should be very agnostic as to how a "version" is indicated - semver, vers, name, hash or anything else.

5.9.1 Versions and TEIs

Each product object has one or multiple TEI URNs.

For the API to be able to present a list of versions in a cronological order, a timestamp for a release is required.

5.9.2 TEA Component Object

A TEA Component object has the following parts:

- **uuid**: A unique identifier for the TEA component
- **name**: Component name

- **identifiers**: List of identifiers for the component
  - **idType**: Type of identifier, e.g. **`tei`**, **`purl`**, **`cpe`**
  - **idValue**: Identifier value

Note: In coming versions, there may be a flag indicating lifecycle status for a component.

5.9.2.1 Examples

Some examples of Maven artefacts as TEA Components:

```
{
  "uuid": "3910e0fd-aff4-48d6-b75f-8bf6b84687f0",
  "name": "Apache Log4j API",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-api"
    }
  ]
}
```

```
{
  "uuid": "b844c9bd-55d6-478c-af59-954a932b6ad3",
  "name": "Apache Log4j Core",
  "identifiers": [
    {
      "idType": "CPE",
      "idValue": "cpe:2.3:a:apache:log4j"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-core"
    }
  ]
}
```

5.9.3 Handling the Pre-Release flag

The "Pre-release" flag is used to indicate that this is not a final release. For a given Component with a UUID, the flag can be set to indicate a "test", "beta", "alpha" or similar non-deployed release. It can only be set when creating the Component. The TEA implementation may allow it to be unset (False) once. This is to support situations where a object is promoted as is after testing to production version. The flag can not be set after initial creation and publication of the Component.

If the final version is different from the pre-release (bugs fixed, code changed, different binary) a new Component with a new UUID and version needs to be created.

5.9.4 References

- Semantic versioning (Semver): https://semver.org <https://semver.org>
- PURL VERS https://github.com/package-url/purl-spec/blob/master/VERSION-RANGE-SPEC.rst

5.10 TEA Component Release

5.10.1 Overview

A TEA Component Release represents a specific version of a TEA Component lineage. It is the concrete, versioned entity which has collections of security-related artefacts (SBOM, VDR/VEX, attestations, etc.).

Key attributes:

- uuid: Unique identifier of the component release
- component: UUID of the TEA Component this release belongs to
- version: Human-readable version string
- createdDate: Timestamp when the release was created in TEA
- releaseDate: Upstream release timestamp
- preRelease: Indicates pre-release/beta status
- identifiers: Array of identifiers (idType: CPE/TEI/PURL; idValue: string)

Collections for a release contain artefacts relevant to that specific release.

5.10.2 JSON examples

The following examples are reused from the OpenAPI schema (**components/schemas/release.examples**), ensuring exact field names and casing.


```
{
  "uuid": "605d0ecb-1057-40e4-9abf-c400b10f0345",
  "version": "11.0.6",
  "createdDate": "2025-04-01T15:43:00Z",
  "releaseDate": "2025-04-01T15:43:00Z",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.6"
    }
  ]
}
```

```
{
  "uuid": "da89e38e-95e7-44ca-aa7d-f3b6b34c7fab",
  "version": "10.1.40",
  "createdDate": "2025-04-01T18:20:00Z",
  "releaseDate": "2025-04-01T18:20:00Z",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@10.1.40"
    }
  ]
}
```

```
{
  "uuid": "95f481df-f760-47f4-b2f2-f8b76d858450",
  "version": "11.0.0-M26",
  "createdDate": "2024-09-13T17:49:00Z",
  "preRelease": true,
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.0-M26"
    }
  ]
}
```

Notes:

- Use uppercase idType values exactly as defined by the schema enum: CPE, TEI, PURL.

5.11 TEA Releases and Collections

5.11.1 The TEA Component Release object (TRO)

A TEA Component Release object represents a specific version of a component, identified by a unique version number and associated metadata. Each release may include multiple distributions, which capture variations such as architecture, packaging, or localization.

- For software components, each distribution typically corresponds to a different digital file delivered to users (e.g., by platform or packaging type).
- For hardware components, distributions may reflect differences in packaging, language, or other physical attributes.

Each distribution is assigned a unique **`distributionType`**, defined by the producer, which is used to associate relevant TEA Artifacts with that distribution. Since TEA Artifacts can be associated with multiple release objects, the taxonomy for **`distributionType`** values should be defined on a TEA service level and consistently applied to all TEA Artifacts published by that producer. This ensures global uniqueness and reliable association across releases.

The **`uuid`** of the TEA Component Release object is identical to the **`uuid`** of its associated TEA Collection object (TCO).

5.11.1.1 Structure

A TEA Component Release object contains the following fields:

- **uuid**: Unique identifier for the TEA Component Release.
- **version**: Version number of the release.
- **createdDate**: Timestamp when the release object was created.
- **releaseDate**: Timestamp of the actual release.
- **preRelease**: Boolean flag indicating if this is a pre-release (e.g., beta). This flag can be disabled after creation, but not enabled.
- **identifiers**: List of identifiers for the component.
- **idType**: Type of identifier (e.g., **`TEI`**, **`PURL`**, **`CPE`**).
- **idValue**: Value of the identifier.
- **distributions**: List of release distributions, each with:
  - **distributionType**: Unique identifier for the distribution type.
  - **description**: Free-text description of the distribution.
  - **identifiers**: List of identifiers specific to this distribution.
  - **idType**: Type of identifier (e.g., **`TEI`**, **`PURL`**, **`CPE`**).

- **idValue**: Value of the identifier.
- **url**: Direct download URL for the distribution.
- **signatureUrl**: Direct download URL for the distribution's external signature.
- **checksums**: List of checksums for the distribution.
  - **algType**: Checksum algorithm used.
  - **algValue**: Checksum value.

5.11.1.2 Examples

5.11.1.2.1 Single distribution

This example shows a TEA Component Release for Apache Log4j Core, which is distributed as a single JAR file. Even though there is only one distribution, the **`distributions`** attribute is included to enable searching for the release by the SHA-256 checksum of the JAR. This structure also allows for future extensibility if additional distributions are introduced.

Example of simple release


```
{
  "uuid": "b1e2c3d4-5678-49ab-9cde-123456789abc",
  "version": "2.24.3",
  "createdDate": "2024-12-10T10:51:00Z",
  "releaseDate": "2024-12-13T12:52:29Z",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-core@2.24.3"
    }
  ],
  "distributions": [
    {
      "distributionType": "jar",
      "description": "Binary distribution",
      "identifiers": [
        {
          "idType": "PURL",
          "idValue": "pkg:maven/org.apache.logging.log4j/log4j-core@2.24.3?type=ja
        }
      ],
      "checksums": [
        {
          "algType": "SHA-256",
          "algValue": "b1e2c3d4f5a67890b1e2c3d4f5a67890b1e2c3d4f5a67890b1e2c3d4f5a
        }
      ],
      "url": "https://repo.maven.apache.org/maven2/org/apache/logging/log4j/log4j-
      "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/logging/log
    }
  ]
}
```

5.11.1.2.2 Multiple distributions

This is an example of a TEA Component Release for Apache Tomcat 11.0.7 binary distributions. The example defines four distinct **`distributionType`**s, which is essential not only for associating the correct SBOMs with each distribution, but also for accurately tracking and reporting vulnerabilities that may affect only specific distributions. For instance:

- The **`zip`** and **`tar.gz`** distributions contain only Java JARs.
- The **`windows-x64.zip`** distribution additionally includes the Apache Procrun <https://commons.apache.org/proper/commons-daemon/procrun.html> binary, which is specific to Windows and may introduce unique vulnerabilities.
- The **`windows-x64.exe`** distribution contains the same data as **`windows-x64.zip`**, but is packaged as a self-extracting installer created by the Nullsoft Scriptable Install System <https://nsis.sourceforge.io/Main_Page>.

By defining separate **`distributionType`**s, it becomes possible to precisely associate artefacts and vulnerability disclosures with the affected distributions, ensuring accurate risk assessment and remediation.

Example of four different binary distributions in the same release

```
{
  "uuid": "605d0ecb-1057-40e4-9abf-c400b10f0345",
  "version": "11.0.7",
  "createdDate": "2025-05-07T18:08:00Z",
  "releaseDate": "2025-05-12T18:08:00Z",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.7"
    }
  ],
  "distributions": [
    {
      "distributionType": "zip",
      "description": "Core binary distribution, zip archive",
      "identifiers": [
        {
          "idType": "PURL",
          "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.7?type=zip"
        }
      ],
      "checksums": [
        {
          "algType": "SHA_256",
          "algValue": "9da736a1cdd27231e70187cbc67398d29ca0b714f885e7032da9f1fb247
        }
      ],
      "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7
      "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomc
    },
    {
      "distributionType": "tar.gz",
      "description": "Core binary distribution, tar.gz archive",
      "identifiers": [
        {
          "idType": "PURL",
          "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.7?type=tar.gz"
        }
      ],
      "checksums": [
        {
          "algType": "SHA_256",
          "algValue": "2fcece641c62ba1f28e1d7b257493151fc44f161fb391015ee6a95fa716
        }
      ],
      "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7
      "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomc
    },
    {
      "distributionType": "windows-x64.zip",
      "description": "Core binary distribution, Windows x64 zip archive",
      "identifiers": [
        {
          "idType": "PURL",
          "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.7?classifier=windows
        }
      ],
      "checksums": [
        {
```

```
            "algType": "SHA_256",
            "algValue": "62a5c358d87a8ef21d7ec1b3b63c9bbb577453dda9c00cbb522b16cee6c
          }
        ],
        "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7
        "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomc
      },
      {
        "distributionType": "windows-x64.exe",
        "description": "Core binary distribution, Windows Service Installer (MSI)",
        "checksums": [
          {
            "algType": "SHA_512",
            "algValue": "1d3824e7643c8aba455ab0bd9e67b14a60f2aaa6aa7775116bce40eb057
          }
        ],
        "url": "https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.7/bin/apache-tomcat-
        "signatureUrl": "https://downloads.apache.org/tomcat/tomcat-11/v11.0.7/bin/a
      }
    ]
  }
```


5.11.1.2.3 Pre-release flag usage

The **preRelease** flag is used to indicate that a release is not production ready, regardless of the version naming scheme. This helps consumers identify non-production releases without relying on conventions like **-beta**, **-rc**, or **-M** in the version string.

There are two main scenarios for using the **preRelease** flag:

- **Pending release:** The distribution is still undergoing quality assurance or review and is not yet officially released. These typically lack a **releaseDate** attribute. Once the release is approved, the **preRelease** flag is set to **false** and the **releaseDate** is added.
- **Permanent pre-release:** The distribution is intentionally marked as a pre-release (e.g., beta, milestone, or release candidate) and will never be considered production ready, even after all checks are complete. These may have a **releaseDate**, but **preRelease** remains **true**.

Examples of non-production ready distributions

- **Pending release (no `releaseDate`):**


```
{
  "uuid": "e2a1c7b4-3f2d-4e8a-9c1a-7b2e4d5f6a8b",
  "version": "11.0.0",
  "createdDate": "2025-09-01T00:00:00Z",
  "preRelease": true,
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.0?repository_url=http
    }
  ]
}
```


- **Transition to production-ready (`preRelease` flag turned off, `releaseDate` added)**:

```
{
  "uuid": "e2a1c7b4-3f2d-4e8a-9c1a-7b2e4d5f6a8b",
  "version": "11.0.0",
  "createdDate": "2025-09-01T00:00:00Z",
  "releaseDate": "2025-09-10T12:00:00Z",
  "preRelease": false,
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.0"
    }
  ]
}
```


- **Beta version (has `releaseDate`, but not production ready)**:


```
{
  "uuid": "95f481df-f760-47f4-b2f2-f8b76d858450",
  "version": "11.0.0-M26",
  "createdDate": "2024-09-13T17:49:00Z",
  "releaseDate": "2024-09-16T17:49:00Z",
  "preRelease": true,
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.0-M26"
    }
  ]
}
```


5.11.2 TEA Collection object (TCO)

For each product and version there is a Tea Collection object, which is a list of available artefacts for this specific version. The TEA Index is a list of TEA collections.

The TEA collection is normally created by the TEA application server at publication time of artefacts. The publisher may sign the collection object as a JSON file at time of publication.

If there are any updates of artefacts within a collection for the same version of a product, then a new TEA Collection object is created and signed. This update will have the same UUID, but a new version number. A reason for the update will have to be provided. This shall be used to correct mistakes, spelling errors as well as to provide new information on dynamic artefact types such as LCE or VEX. If the product is modified, that is a new product version and that should generate a new collection object with a new UUID and updated metadata.

The API allows for retrieving the latest version of the collection, or a specific version.

5.11.2.1 Dynamic or static Collection objects

The TCO is produced by the TEA software platform. There are two ways to implement this:

- **Dynamic**: The TCO is built for each API request and created dynamically.
- **Static**: The TCO is built at publication time as a static object by the publisher. This object can be digitally signed at publication time and version controlled.

5.11.2.2 Collection object

The TEA Collection object has the following parts:

- Preamble
- **uuid**: UUID of the TEA Collection object. Note that this is equal to the UUID of the associated TEA Component Release object. When updating a collection, only the **`version`** is changed.
- **version**: TEA Collection version, incremented each time its content changes. Versions start with 1.
- **date**: TEA Collection version release date.
- **belongsTo**: Scope of the collection; enum values **`RELEASE`** or **`PRODUCT_RELEASE`**.
- **updateReason**: Reason for the update/release of the TEA Collection object.
  - **type**: Type of update reason. See reasons for TEA Collection update below.
  - **comment**: Free text description.
-
- **artifacts**: List of TEA Artifact objects. See below.

5.11.3 TEA Artifact object

A TEA Artifact object represents a security-related document or file linked to a component release, such as an SBOM, VEX, attestation, or license. TEA Artifacts are strictly **immutable**: if the underlying document changes, a new TEA Artifact object must be created. URLs referenced in this object must always resolve to the same resource to ensure that published checksums remain valid and verifiable.

TEA Artifacts can be reused across multiple TEA Collections, allowing the same document to be referenced by different component or product releases. This promotes consistency and reduces duplication.

Optionally, each TEA Artifact can specify the **`distributionType`** identifiers of the distributions it applies to. If this field is absent, the TEA Artifact is considered applicable to all distributions of the release.

5.11.3.1 Structure

A TEA Artifact object contains the following fields:

- **uuid**: The UUID of the TEA Artifact object. Together with *version* uniquely identifies the TEA Artifact.
- **version**: An integer with default value 1. Together with *uuid* uniquely identifies the TEA Artifact. This field can be used to designate successive, immutable revisions of an artefact content (e.g. an updated VEX file).
- **name**: A human-readable name for the artefact.
- **type**: The type of artefact. See TEA Artifact types for allowed values (e.g., **`BOM`**, **`VULNERABILITIES`**, **`LICENSE`**).
- **createdDate**: The date and time the TEA Artefact revision was created.
- **componentDistributions** (optional):
  An array of **`distributionType`** identifiers indicating which distributions this TEA Artifact applies to. If omitted, the TEA Artifact applies to all distributions.
- **formats**:
  An array of objects, each representing the same artefact content in a different format. The order of the list is not significant. Each format object includes:
  - **mediaType**: The MIME type of the document (e.g., **`application/vnd.cyclonedx+xml`**).
  - **description**: A free-text description of the artefact format.
  - **url**: A direct download URL for the artefact. This must point to an immutable resource.
  - **signatureUrl** (optional): A direct download URL for a detached digital signature of the artefact, if available.
  - **checksums**:
    An array of checksum objects for the artefact, each containing:
    - **algType**: The checksum algorithm used (e.g., **`SHA_256`**, **`SHA3_512`**).
    - **algValue**: The checksum value as a string.

5.11.3.2 Notes

- The **`formats`** array allows the same artefact to be provided in multiple encodings or serializations (e.g., JSON, XML).
- The **`checksums`** field provides integrity verification for each artefact format.
- The **`signatureUrl`** enables consumers to verify the authenticity of the artefact using detached signatures.
- Artefacts should be published to stable, versioned URLs to ensure immutability and traceability.

5.11.4 The reason for TCO update enum

ENUM	Description
INITIAL_RELEASE	Initial release of the collection
VEX_UPDATED	Updated the VEX artefact(s)
ARTIFACT_UPDATED	Updated the artefact(s) other than VEX
ARTIFACT_REMOVED	Removal of artefact
ARTIFACT_ADDED	Addition of an artefact

Updates of VEX (CSAF) files may be handled in a different way by a TEA client, producing different alerts than other changes of a collection.

5.11.5 TEA Artifact types

ENUM	Description
ATTESTATION	Machine-readable statements containing facts, evidence, or testimony.
BOM	Bill of Materials: SBOM, OBOM, HBOM, SaaSBOM, etc.
BUILD_META	Build-system specific metadata file: **`pom.xml`**, **`package.json`**, **`.nuspec`**, etc.
CERTIFICATION	Industry, regulatory, or other certification from an accredited certification body.
FORMULATION	Describes how a component or service was manufactured or deployed.
LICENSE	License file
RELEASE_NOTES	Release notes document
SECURITY_TXT	A **`security.txt`** file
THREAT_MODEL	A threat model
VULNERABILITIES	A list of vulnerabilities: VDR/VEX
OTHER	Document that does not fall into any of the above categories

5.11.5.1 Examples


```
{
  "uuid": "4c72fe22-9d83-4c2f-8eba-d6db484f32c8",
  "version": 10,
  "date": "2024-12-13T00:00:00Z",
  "updateReason": {
    "type": "ARTIFACT_UPDATED",
    "comment": "VDR file updated"
  },
  "artifacts": [
    {
      "uuid": "1cb47b95-8bf8-3bad-a5a4-0d54d86e10ce",
      "name": "Build SBOM",
      "type": "BOM",
      "formats": [
        {
          "mediaType": "application/vnd.cyclonedx+xml",
          "description": "CycloneDX SBOM (XML)",
          "url": "https://repo.maven.apache.org/maven2/org/apache/logging/log4j/lo
          "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/logging
          "checksums": [
            {
              "algType": "MD5",
              "algValue": "2e1a525afc81b0a8ecff114b8b743de9"
            },
            {
              "algType": "SHA-1",
              "algValue": "5a7d4caef63c5c5ccdf07c39337323529eb5a770"
            }
          ]
        }
      ]
    },
    {
      "uuid": "dfa35519-9734-4259-bba1-3e825cf4be06",
      "version": 7,
      "name": "Vulnerability Disclosure Report",
      "type": "VULNERABILITIES",
      "formats": [
        {
          "mediaType": "application/vnd.cyclonedx+xml",
          "description": "CycloneDX VDR (XML)",
          "url": "https://logging.apache.org/cyclonedx/vdr.xml",
          "checksums": [
            {
              "algType": "SHA-256",
              "algValue": "75b81020b3917cb682b1a7605ade431e062f7a4c01a412f0b87543b
            }
          ]
        }
      ]
    }
  ]
}
```


5.12 Known types

meta:enum:

- vcs: Version Control System
- issue-tracker: Issue or defect tracking system, or an Application Lifecycle Management (ALM) system
- website: Website
- advisories: Security advisories
- bom: Bill of Materials (SBOM, OBOM, HBOM, SaaSBOM, etc)
- mailing-list: Mailing list or discussion group
- social: Social media account
- chat: Real-time chat platform
- documentation: Documentation, guides, or how-to instructions
- support: Community or commercial support
- source-distribution: description: The location where the source code distributable can be obtained. This is often an archive format such as zip or tgz. The source-distribution type complements use of the version control (vcs) type.
- distribution: Direct or repository download location
- distribution-intake: The location where a component was published to. This is often the same as "distribution" but may also include specialized publishing processes that act as an intermediary.
- license: The reference to the license file. If a license URL has been defined in the license node, it should also be defined as an external reference for completeness.
- build-meta: Build-system specific meta file (i.e. pom.xml, package.json, .nuspec, etc)
- build-system: Reference to an automated build system
- release-notes: Reference to release notes
- security-contact: Specifies a way to contact the maintainer, supplier, or provider in the event of a security incident. Common URIs include links to a disclosure procedure, a mailto (RFC-2368) that specifies an email address, a tel (RFC-3966) that specifies a phone number, or dns (RFC-4501) that specifies the records containing DNS Security TXT.
- model-card: A model card describes the intended uses of a machine learning model, potential limitations, biases, ethical considerations, training parameters, datasets used to train the model, performance metrics, and other relevant data useful for ML transparency.
- log: A record of events that occurred in a computer system or application, such as problems, errors, or information on current operations. configuration: Parameters or settings that may be used by other components or services.
- evidence: Information used to substantiate a claim.
- formulation: Describes how a component or service was manufactured or deployed.
- attestation: Human or machine-readable statements containing facts, evidence, or testimony.
- threat-model: An enumeration of identified weaknesses, threats, and countermeasures, dataflow diagram (DFD), attack tree, and other supporting documentation in human-readable or machine-readable format.
- adversary-model: The defined assumptions, goals, and capabilities of an adversary.
- risk-assessment: Identifies and analyzes the potential of future events that may negatively impact individuals, assets, and/or the environment. Risk assessments may also include judgments on the tolerability of each risk.
- vulnerability-assertion: A Vulnerability Disclosure Report (VDR) which asserts the known and previously unknown vulnerabilities that affect a component, service, or product including the analysis and findings describing the impact (or lack of impact) that the reported vulnerability has on a component, service, or product.
- exploitability-statement: A Vulnerability Exploitability eXchange (VEX) which asserts the known vulnerabilities that do not affect a product, product family, or organization, and optionally the ones that do. The VEX should include the analysis and findings describing the impact (or lack of impact) that the reported vulnerability has on the product, product family, or organization.
- pentest-report: Results from an authorized simulated cyberattack on a component or service, otherwise known as a penetration test.
- static-analysis-report: SARIF or proprietary machine or human-readable report for which static analysis has identified code quality, security, and other potential issues with the source code.
- dynamic-analysis-report: Dynamic analysis report that has identified issues such as vulnerabilities and misconfigurations.
- runtime-analysis-report: Report generated by analyzing the call stack of a running application.
- component-analysis-report: Report generated by Software Composition Analysis (SCA), container analysis, or other forms of component analysis.
- maturity-report: Report containing a formal assessment of an organization, business unit, or

team against a maturity model.
- certification-report: Industry, regulatory, or other certification from an accredited (if applicable) certification body.
- codified-infrastructure: Code or configuration that defines and provisions virtualized infrastructure, commonly referred to as Infrastructure as Code (IaC).
- quality-metrics: Report or system in which quality metrics can be obtained.
- poam: Plans of Action and Milestones (POAM) complement an "attestation" external reference. POAM is defined by NIST as a "document that identifies tasks needing to be accomplished. It details resources required to accomplish the elements of the plan, any milestones in meeting the tasks and scheduled completion dates for the milestones".
- electronic-signature: An e-signature is commonly a scanned representation of a written signature or a stylized script of the person's name.
- digital-signature: A signature that leverages cryptography, typically public/private key pairs, which provides strong authenticity verification.
- rfc-9116: Document that complies with RFC-9116 (A File Format to Aid in Security Vulnerability Disclosure)
- other: Use this if no other types accurately describe the purpose of the external reference.

5.13 Transparency Exchange API - Trusting digital signatures in TEA

Software transparency requires a trust platform so that users can validate the information and artefacts published. Given the situation today any information published is better than none, so the framework for digital signatures will not be mandatory for API compliance. Implementations may require all published information to be signed and validated. In some vertical markets branch standards may require digital signatures.

Within the TEA API documents may be signed with an electronic signature. CycloneDX Documents support signatures <https://cyclonedx.org/use-cases/#authenticity> within the JSON and XML files, but other artefacts may need external signature files, a detached signature.

5.13.1 Requirements

Digital signatures provide integrity and identity to published data.

- **Integrity**: Documents dowloaded must be the same as documents published
- **Identity**: Customers need to be able to verify the publisher of the documents and verify that it is the expected publisher. A TEA server may want to verify that published documents are signed by the expected publisher and that signatures are valid.

In order to sign an object, a pair of asymmetric keys will be needed. The public key is used to create a certificate, signed by a certificate authority (CA). The private key is used for signing and needs to be protected.

A software publisher may buy CA services from a commercial vendor or set up an internal PKI solution. The issue with internal PKIs is that external parties do not automatically trust that internal PKI.

This document outlines a proposal on how to build that trust and make it possible for publishers to use an internal PKI. It is of course important that this PKI is maintained according to best current practise.

5.13.2 API trust

The TEA API is built on the HTTP protocol with TLS encryption and authentication, using the **`https://`** URL scheme.

The TLS server certificate is normally issued by a public Certificate Authority that is part of the Web PKI. The client needs to validate the TLS server certificate to make sure

- that the certificate name (CN or Subject Alt Name) matches the host part of the URI.

- that the certificate is valid, i.e. the not-before date and the not-after date is not out of range
- that the certificate is signed by a trusted CA

If the certificate validates properly, the API can be trusted. Validation proves that the server is the right server for the given host name in the URL.

This trust can be used to implement trust in a private PKI used to sign documents delivered over the API.

In addition, trust anchors can be published in DNSsec as an extra level of validation.

5.13.3 Getting trust anchors

Much like the EST protocol, the TEA protocol can be used to download trust anchors for a private PKI. These are PEM-encoded certificates in one single text file.

The TEA API has a **`/trust-anchors/`** API that will download the current trust anchor APIs. This file is not signed, that would cause a chicken-and-egg problem. The certificates in the file are all signed.

An implementation should download these and apply them only for this service, not in global scope. A PKI valid for example.com <http://example.com> is not valid for example.net <http://example.net>.

Note that the TEA api can host many years of documents for published versions. Old and expired trust anchors may be needed to validate digital signatures on old documents.

5.13.4 Validating the trust anchors using DNSsec (DANE)

5.13.5 Digital signatures

5.13.5.1 Digital signatures as specified for CycloneDX

> "Digital signatures may be applied to a BOM or to an assembly within a BOM. CycloneDX supports XML Signature, JSON Web Signature (JWS), and JSON Signature Format (JSF). Signed BOMs benefit by providing advanced integrity and non-repudiation capabilities." https://cyclonedx.org/use-cases/#authenticity

5.13.5.2 External (detached) digital signatures for other documents

- indication of hash algorithm
- indicator of cert used
- intermediate cert and sign cert

5.13.5.3 Validating the digital signature

5.13.6 Using Sigstore for signing

Sigstore is an excellent free service for both signing of GIT commits as well as artefacts by using ephemeral certificates (very shortlived) and a certificate transparency log for validation and verification. Sigstore signatures contain timestamps from a timestamping service.

Sigstore lends itself very well to Open Source projects but not really commercial projects. The Sigstore platform can be deployed internally for enterprise use, but in that case will have the same problem as any internal PKI with establishing trust.

5.13.7 Suggested PKI setup

5.13.7.1 Root cert

5.13.7.1.1 Root cert validity and renewal

5.13.7.2 Intermediate cert

5.13.7.2.1 Intermediate cert validity and renewal

5.13.7.3 Signature

5.13.7.3.1 Time stamp services

5.13.7.4 DNS entry

5.13.8 References

- IETF RFC DANE
- IETF DANCE architecture (IETF draft)
- IETF Digital signature
- JSON web signatures (JWS) - https://datatracker.ietf.org/doc/html/rfc7515
- JSON signature format (JSF) - https://cyberphone.github.io/doc/security/jsf.html
- IETF Enrollment over Secure Transport (EST) RFC 7030 <https://www.rfc-editor.org/rfc/rfc7030>

6 The TEA API

6.1 CLE

6.1.1 getCleByProductReleaseId

GET `/productRelease/{uuid}/cle`

Get the CLE (Common Lifecycle Enumeration) data for a TEA Product Release

Table 1: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Product Release in the TEA server

Table 2: Responses

Status	Description	Schema
200	CLE data for the requested TEA Product Release found and returned	**cle**
400		—
404		—

6.1.2 getCleByProductId

GET `/product/{uuid}/cle`

Get the CLE (Common Lifecycle Enumeration) data for a TEA Product

Table 3: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Product in the TEA server

Table 4: Responses

Status	Description	Schema
200	CLE data for the requested TEA Product found and returned	**cle**
400		—
404		—

6.1.3 getCleByComponentId

GET `/component/{uuid}/cle`

Get the CLE (Common Lifecycle Enumeration) data for a TEA Component

Table 5: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Component in the TEA server

Table 6: Responses

Status	Description	Schema
200	CLE data for the requested TEA Component found and returned	**cle**
400		—
404		—

6.1.4 getCleByComponentReleaseId

GET `/componentRelease/{uuid}/cle`

Get the CLE (Common Lifecycle Enumeration) data for a TEA Component Release

Table 7: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Component Release in the TEA server

Table 8: Responses

Status	Description	Schema
200	CLE data for the requested TEA Component Release found and returned	cle
400		—
404		—

6.2 TEA Artifact

6.2.1 getLatestArtifact

GET `/artifact/{uuid}/latest`

Get metadata for latest revision of a specific TEA Artifact

Table 9: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Artifact in the TEA server

Table 10: Responses

Status	Description	Schema
200	Requested TEA Artifact metadata found and returned	artifact
400		—
404		—

6.2.2 getArtifactByVersion

GET `/artifact/{uuid}/{artifactVersion}`

Get metadata for a specific revision of a specific TEA Artifact

Table 11: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Artifact in the TEA server
artifactVersion	path	Yes	integer	Version of TEA Artifact

Table 12: Responses

Status	Description	Schema
200	Requested TEA Artifact metadata found and returned	artifact
400		—
404		—

6.3 TEA Component

6.3.1 getTeaComponentById

GET `/component/{uuid}`

Get a TEA Component

Table 13: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Component in the TEA server

Table 14: Responses

Status	Description	Schema
200	Requested TEA Component found and returned	component
400		—
404		—

6.3.2 getReleasesByComponentId

GET `/component/{uuid}/releases`

Get releases of the component

Table 15: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Component in the TEA server

Table 16: Responses

Status	Description	Schema
200	Requested Releases of TEA Component found and returned	Array of release
400		—
404		—

6.3.3 queryTeaComponents

GET `/components`

Returns a list of TEA components. Note that multiple components may match.

Table 17: Parameters

Name	In	Required	Type	Description
		No	—	
		No	—	
		No	—	
		No	—	

Table 18: Responses

Status	Description	Schema
200		—
400		—

6.4 TEA Component Release

6.4.1 queryTeaComponentReleases

GET `/componentReleases`

Returns a list of TEA component releases. Note that multiple component releases may match.

Table 19: Parameters

Name	In	Required	Type	Description
		No	—	
		No	—	
		No	—	
		No	—	

Table 20: Responses

Status	Description	Schema
200		—
400		—

6.4.2 getComponentReleaseById

GET `/componentRelease/{uuid}`

Get the TEA Component Release with its latest collection

Table 21: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Component Release in the TEA server

Table 22: Responses

Status	Description	Schema
200	Requested TEA Component Release and its latest Collection found and returned	**component-release-with-collection**
400		—
404		—

6.4.3 getLatestCollection

GET `/componentRelease/{uuid}/collection/latest`

Get the latest TEA Collection belonging to the TEA Component Release

Table 23: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Component Release in the TEA server

Table 24: Responses

Status	Description	Schema
200	Requested TEA Collection found and returned	**collection**
400		—
404		—

6.4.4 getCollectionsByReleaseId

GET `/componentRelease/{uuid}/collections`

Get the TEA Collections belonging to the TEA Component Release

Table 25: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Component Release in the TEA server

Table 26: Responses

Status	Description	Schema
200	Requested TEA Collection found and returned	Array of **collection**
400		—
404		—

6.4.5 getCollection

GET **/componentRelease/{uuid}/collection/{collectionVersion}**

Get a specific Collection (by version) for a TEA Component Release by its UUID

Table 27: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Collection in the TEA server
collectionVersion	path	Yes	integer	Version of TEA Collection

Table 28: Responses

Status	Description	Schema
200	Requested TEA Collection Version found and returned	**collection**
400		—
404		—

6.5 TEA Discovery

6.5.1 discoveryByTei

GET **/discovery**

Discovery endpoint which resolves TEI into product release UUID.

Table 29: Parameters

Name	In	Required	Type	Description
tei	query	Yes	string	Transparency Exchange Identifier (TEI) for the product being discovered. Provide the TEI as a URL-encoded string per RFC 3986, RFC 3987.

Table 30: Responses

Status	Description	Schema
200		—
400		—
404		—

6.6 TEA Product

6.6.1 getTeaProductByUuid

GET `/product/{uuid}`

Get a TEA Product by UUID

Table 31: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of the TEA product in the TEA server

Table 32: Responses

Status	Description	Schema
200	Requested TEA Product found and returned	product
400		—
404		—

6.6.2 queryTeaProducts

GET `/products`

Returns a list of TEA products. Note that multiple products may match.

Table 33: Parameters

Name	In	Required	Type	Description
		No	—	
		No	—	
		No	—	
		No	—	

Table 34: Responses

Status	Description	Schema
200		—
400		—

6.7 TEA Product Release

6.7.1 getReleasesByProductId

GET `/product/{uuid}/releases`

Get releases of the product

Table 35: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Product in the TEA server
		No	—	
		No	—	

Table 36: Responses

Status	Description	Schema
200		—
400		—
404		—

6.7.2 getTeaProductReleaseByUuid

GET `/productRelease/{uuid}`

Get a TEA Product Release

Table 37: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Product Release in the TEA server

Table 38: Responses

Status	Description	Schema
200	Requested TEA Product Release found and returned	productRelease
400		—
404		—

6.7.3 queryTeaProductReleases

GET `/productReleases`

Returns a list of TEA product releases. Note that multiple product releases may match.

Table 39: Parameters

Name	In	Required	Type	Description
		No	—	
		No	—	
		No	—	
		No	—	

Table 40: Responses

Status	Description	Schema
200		—
400		—

6.7.4 getLatestCollectionForProductRelease

GET `/productRelease/{uuid}/collection/latest`

Get the latest TEA Collection belonging to the TEA Product Release

Table 41: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	uuid	UUID of TEA Product Release in the TEA server

Table 42: Responses

Status	Description	Schema
200	Requested TEA Collection found and returned	collection
400		—
404		—

6.7.5 getCollectionsByProductReleaseId

GET `/productRelease/{uuid}/collections`

Get the TEA Collections belonging to the TEA Product Release

Table 43: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Product Release in the TEA server

Table 44: Responses

Status	Description	Schema
200	Requested TEA Collection found and returned	Array of **collection**
400		—
404		—

6.7.6 getCollectionForProductRelease

GET `/productRelease/{uuid}/collection/{collectionVersion}`

Get a specific Collection (by version) for a TEA Product Release by its UUID

Table 45: Parameters

Name	In	Required	Type	Description
uuid	path	Yes	**uuid**	UUID of TEA Product Release in the TEA server
collectionVersion	path	Yes	integer	Version of TEA Collection

Table 46: Responses

Status	Description	Schema
200	Requested TEA Collection Version found and returned	**collection**
400		—
404		—

7 Data model

This section catalogs the named schemas declared in **components.schemas**.

7.1 artifact

A security-related document

Type: object

Table 47: Properties

Property	Type	Requirement	Description
uuid	**uuid**	Required	The UUID of the TEA Artifact object. Together with **version** uniquely identifies the TEA Artifact.
version	integer	Optional	An integer with default value 1. Together with **uuid** uniquely identifies the TEA Artifact. This field can be used to designate successive, immutable revisions of an artefact content (e.g. an updated VEX file).
name	string	Optional	Name of TEA Artifact
type	**artifact-type**	Required	Type of TEA Artifact
createdDate	**date-time**	Optional	The date when the TEA Artifact revision was created.
distributionIds	Array of **uuid**	Optional	List of TEA Component Release distributions that this TEA Artifact applies to. If absent or empty, the TEA Artifact applies to all distributions.
formats	Array of **artifact-format**	Required	List of objects with the same content, but in different formats. The order of the list has no significance.

Example 1 (Informative)

```
{
  "uuid": "1cb47b95-8bf8-3bad-a5a4-0d54d86e10ce",
  "name": "Build SBOM",
  "type": "BOM",
  "formats": [
    {
      "mediaType": "application/vnd.cyclonedx+xml",
      "description": "CycloneDX SBOM (XML)",
      "url": "https://repo.maven.apache.org/maven2/org/apache/logging/
log4j/log4j-core/2.24.3/log4j-core-2.24.3-cyclonedx.xml",
      "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/
logging/log4j/log4j-core/2.24.3/log4j-core-2.24.3-cyclonedx.xml.asc",
      "checksums": [
        {
          "algType": "MD5",
          "algValue": "2e1a525afc81b0a8ecff114b8b743de9"
        },
        {
          "algType": "SHA-1",
          "algValue": "5a7d4caef63c5c5ccdf07c39337323529eb5a770"
        }
      ]
    }
  ]
}
```

Example 2 (Informative)

```
{
  "uuid": "dfa35519-9734-4259-bba1-3e825cf4be06",
  "version": 7,
  "name": "Vulnerability Disclosure Report",
  "type": "VULNERABILITIES",
  "formats": [
    {
      "mediaType": "application/vnd.cyclonedx+xml",
      "description": "CycloneDX VDR (XML)",
      "url": "https://logging.apache.org/cyclonedx/vdr.xml",
      "checksums": [
        {
          "algType": "SHA-256",

"algValue":
"75b81020b3917cb682b1a7605ade431e062f7a4c01a412f0b87543b6e995ad2a"
        }
      ]
    }
  ]
}
```

7.2 artifact-format

A security-related document in a specific format

Type: object

Table 48: Properties

Property	Type	Requirement	Description
mediaType	string	Optional	The Media Type of the document
description	string	Optional	A free text describing the TEA Artifact
url	string	Optional	Direct download URL for the TEA Artifact
signatureUrl	string	Optional	Direct download URL for an external signature of the TEA Artifact
checksums	Array of **checksum**	Optional	List of checksums for the TEA Artifact

7.3 artifact-type

Specifies the type of external reference.

Type: string

Table 49: Enumeration of possible values

Value
ATTESTATION

Table 49: Enumeration of possible values *(continued)*

Value
`BOM`
`BUILD_META`
`CERTIFICATION`
`FORMULATION`
`LICENSE`
`RELEASE_NOTES`
`SECURITY_TXT`
`THREAT_MODEL`
`VULNERABILITIES`
`OTHER`

7.4 checksum

Type: object

Table 50: Properties

Property	Type	Requirement	Description
`algType`	`checksum-type`	Required	Checksum algorithm
`algValue`	string	Required	Checksum value

7.5 checksum-type

Checksum algorithm

Type: string

Table 51: Enumeration of possible values

Value
`MD5`
`SHA-1`
`SHA-256`
`SHA-384`

Table 51: Enumeration of possible values *(continued)*

Value
`SHA-512`
`SHA3-256`
`SHA3-384`
`SHA3-512`
`BLAKE2b-256`
`BLAKE2b-384`
`BLAKE2b-512`
`BLAKE3`

7.6 cle

Common Lifecycle Enumeration (CLE) object based on ECMA-428 TC54 TG3 CLE Specification v1.0.0. Contains lifecycle events and optional reusable definitions for a component or product.

Type: object

Table 52: Properties

Property	Type	Requirement	Description
`events`	Array of `cle-event`	Required	Ordered array of CLE Event objects representing lifecycle events. MUST be ordered by ID in descending order (newest events with highest IDs first).
`definitions`	`cle-definitions`	Optional	Container for reusable policy definitions referenced by events

Example (Informative)

```
{
  "events": [
    {
      "id": 3,
      "type": "endOfSupport",
      "effective": "2025-06-01T00:00:00Z",
      "versions": [
        {
          "range": "vers:npm/>=1.0.0|<2.0.0"
        }
      ],
      "supportId": "standard",
      "published": "2025-01-01T00:00:00Z"
    },
    {
      "id": 2,
      "type": "endOfDevelopment",
      "effective": "2025-01-01T00:00:00Z",
      "versions": [
        {
          "version": "1.0.0"
        }
      ],
      "supportId": "standard",
      "published": "2024-06-01T00:00:00Z"
    },
    {
      "id": 1,
      "type": "released",
      "effective": "2024-01-01T00:00:00Z",
      "version": "1.0.0",
      "license": "Apache-2.0",
      "published": "2024-01-01T00:00:00Z"
    }
  ],
  "definitions": {
    "support": [
      {
        "id": "standard",
        "description": "Standard product support policy",
        "url": "https://example.com/support/standard"
      }
    ]
  }
}
```

7.7 cle-definitions

Container for reusable CLE policy definitions

Type: object

Table 53: Properties

Property	Type	Requirement	Description
support	Array of **cle-support-definition**	Optional	List of support policies

7.8 cle-event

A discrete lifecycle event from the CLE specification

Type: object

Table 54: Properties

Property	Type	Requirement	Description
id	integer	Required	A unique, auto-incrementing integer identifier for the event
type	**cle-event-type**	Required	The type of lifecycle event
effective	string	Required	ISO 8601 timestamp (UTC) when the event takes effect
published	string	Required	ISO 8601 timestamp (UTC) when the event was first published
version	string	Optional	Version string (used by released event type)
versions	Array of **cle-version-specifier**	Optional	List of version specifiers affected by this event
supportId	string	Optional	Reference to a support policy ID defined in the definitions section
license	string	Optional	License identifier (used by released event type)
supersededByVersion	string	Optional	Version string that supersedes the affected versions (used by supersededBy event type)
identifiers	Array of **identifier**	Optional	New identifiers for the component (used by componentRenamed event type)
eventId	integer	Optional	ID of the event being withdrawn (used by withdrawn event type)
reason	string	Optional	Human-readable explanation (used by withdrawn event type)
description	string	Optional	Human-readable description of the event
references	Array of string	Optional	List of URLs to supporting documentation

Example 1 (Informative)

```
{
  "id": 1,
  "type": "released",
  "effective": "2024-01-01T00:00:00Z",
  "version": "1.0.0",
  "license": "MIT",
  "published": "2023-06-01T00:00:00Z"
}
```


Example 2 (Informative)


```
{
  "id": 3,
  "type": "endOfSupport",
  "effective": "2024-01-01T00:00:00Z",
  "versions": [
    {
      "version": "1.0.0"
    }
  ],
  "supportId": "standard",
  "published": "2023-06-01T00:00:00Z"
}
```


7.9 cle-event-type

The type of CLE lifecycle event

Type: string

Table 55: Enumeration of possible values

Value
released
endOfDevelopment
endOfSupport
endOfLife
endOfDistribution
endOfMarketing
supersededBy
componentRenamed
withdrawn

7.10 cle-support-definition

A support policy definition from CLE

Type: object

Table 56: Properties

Property	Type	Requirement	Description
id	string	Required	Unique identifier for the support policy
description	string	Required	Human-readable description of the policy
url	string	Optional	URL to detailed documentation about this support policy

Example (Informative)

```
{
  "id": "standard",
  "description": "Standard product support policy",
  "url": "https://example.com/support/standard"
}
```

7.11 cle-version-specifier

A version specifier that can be either a single version or a version range

Type: object

Table 57: Properties

Property	Type	Requirement	Description
version	string	Optional	A specific version string
range	string	Optional	A version range in vers format (e.g. "vers:npm/>=1.0.0|<2.0.0")

7.12 collection

A collection of security-related documents

Type: object

Table 58: Properties

Property	Type	Requirement	Description
uuid	**uuid**	Optional	UUID of the TEA Collection object. This matches the UUID of the associated TEA Component Release or TEA Product Release object. When updating a collection, only the **version** is changed.
version	integer	Optional	TEA Collection version, incremented each time its content changes. Versions start with 1.
date	**date-time**	Optional	The date when the TEA Collection version was created.
belongsTo	**collection-belongs-to-type**	Optional	Indicates whether this collection belongs to a Component Release or a Product Release

Table 58: Properties *(continued)*

Property	Type	Requirement	Description
updateReason	**collection-update-reason**	Optional	Reason for the update/release of the TEA Collection object.
artifacts	Array of **artifact**	Optional	List of TEA Artifact objects.

Example (Informative)

```
{
  "uuid": "4c72fe22-9d83-4c2f-8eba-d6db484f32c8",
  "version": 10,
  "date": "2024-12-13T00:00:00.000Z",
  "updateReason": {
    "type": "ARTIFACT_UPDATED",
    "comment": "VDR file updated"
  },
  "artifacts": [
    {
      "uuid": "1cb47b95-8bf8-3bad-a5a4-0d54d86e10ce",
      "name": "Build SBOM",
      "type": "BOM",
      "formats": [
        {
          "mediaType": "application/vnd.cyclonedx+xml",
          "description": "CycloneDX SBOM (XML)",
          "url": "https://repo.maven.apache.org/maven2/org/apache/
logging/log4j/log4j-core/2.24.3/log4j-core-2.24.3-cyclonedx.xml",
          "signatureUrl": "https://repo.maven.apache.org/maven2/org/
apache/logging/log4j/log4j-core/2.24.3/log4j-
core-2.24.3-cyclonedx.xml.asc",
          "checksums": [
            {
              "algType": "MD5",
              "algValue": "2e1a525afc81b0a8ecff114b8b743de9"
            },
            {
              "algType": "SHA-1",
              "algValue": "5a7d4caef63c5c5ccdf07c39337323529eb5a770"
            }
          ]
        }
      ]
    },
    {
      "uuid": "dfa35519-9734-4259-bba1-3e825cf4be06",
      "version": 7,
      "name": "Vulnerability Disclosure Report",
      "type": "VULNERABILITIES",
      "formats": [
        {
          "mediaType": "application/vnd.cyclonedx+xml",
          "description": "CycloneDX VDR (XML)",
          "url": "https://logging.apache.org/cyclonedx/vdr.xml",
          "checksums": [
            {
              "algType": "SHA-256",
              "algValue":
"75b81020b3917cb682b1a7605ade431e062f7a4c01a412f0b87543b6e995ad2a"
            }
          ]
        }
      ]
    }
  ]
}
```

7.13 collection-belongs-to-type

Indicates whether a collection belongs to a component release or a product release

Type: string

Table 59: Enumeration of possible values

Value
`COMPONENT_RELEASE`
`PRODUCT_RELEASE`

7.14 collection-update-reason

Reason for the update to the TEA collection

Type: object

Table 60: Properties

Property	Type	Requirement	Description
type	collection-update-reason-type	Optional	Type of update reason.
comment	string	Optional	Free text description

7.15 collection-update-reason-type

Type of TEA collection update

Type: string

Table 61: Enumeration of possible values

Value
`INITIAL_RELEASE`
`VEX_UPDATED`
`ARTIFACT_UPDATED`
`ARTIFACT_ADDED`
`ARTIFACT_REMOVED`

7.16 compliance-document-type

Well-known compliance document types. When idType is COMPLIANCE_DOCUMENT, the idValue SHOULD be one of these values.

Type: string

Table 62: Enumeration of possible values

Value
SOC_2_TYPE_I
SOC_2_TYPE_II
SOC_3
ISO_27001
ISO_27017
ISO_27018
ISO_27701
ISO_42001
PCI_DSS
HIPAA
FedRAMP
GDPR
CSA_STAR
NIST_800_53
NIST_800_171
CMMC
HITRUST
TISAX
CYBER_ESSENTIALS
CYBER_ESSENTIALS_PLUS

7.17 component

A TEA component

Type: object

Table 63: Properties

Property	Type	Requirement	Description
uuid	uuid	Required	A unique identifier for the TEA component
name	string	Required	Component name
identifiers	Array of identifier	Required	List of identifiers for the component

Example 1 (Informative)

```
{
  "uuid": "3910e0fd-aff4-48d6-b75f-8bf6b84687f0",
  "name": "Apache Log4j API",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-api"
    }
  ]
}
```

Example 2 (Informative)

```
{
  "uuid": "b844c9bd-55d6-478c-af59-954a932b6ad3",
  "name": "Apache Log4j Core",
  "identifiers": [
    {
      "idType": "CPE",
      "idValue": "cpe:2.3:a:apache:log4j"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-core"
    }
  ]
}
```

7.18 component-ref

A reference to a TEA component or specific component release

Type: object

Table 64: Properties

Property	Type	Requirement	Description
uuid	uuid	Required	A unique identifier for the TEA component
release	uuid	Optional	Optional UUID of a specific release included in the product in the case where the product always include a specific release of a component. The product name should include a version identifier in this case.

7.19 component-release-with-collection

A TEA Component Release combined with its latest collection

Type: object

Table 65: Properties

Property	Type	Requirement	Description
release	release	Required	The TEA Component Release information
latestCollection	collection	Required	The latest TEA Collection for this component release

Example (Informative)

```
{
  "release": {
    "uuid": "605d0ecb-1057-40e4-9abf-c400b10f0345",
    "version": "11.0.7",
    "createdDate": "2025-05-07T18:08:00.000Z",
    "releaseDate": "2025-05-12T18:08:00.000Z",
    "identifiers": [
      {
        "idType": "PURL",
        "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.7"
      }
    ]
  },
  "latestCollection": {
    "uuid": "605d0ecb-1057-40e4-9abf-c400b10f0345",
    "version": 2,
    "date": "2025-05-12T18:08:00.000Z",
    "belongsTo": "COMPONENT_RELEASE",
    "updateReason": {
      "type": "INITIAL_RELEASE",
      "comment": "Initial collection for this release"
    },
    "artifacts": [
      {
        "uuid": "1cb47b95-8bf8-3bad-a5a4-0d54d86e10ce",
        "name": "Build SBOM",
        "type": "BOM",
        "formats": [
          {
            "mediaType": "application/vnd.cyclonedx+xml",
            "description": "CycloneDX SBOM (XML)",
            "url": "https://repo.maven.apache.org/maven2/org/apache/
tomcat/tomcat/11.0.7/tomcat-11.0.7-cyclonedx.xml",
            "checksums": [
              {
                "algType": "SHA-256",
                "algValue":
"9da736a1cdd27231e70187cbc67398d29ca0b714f885e7032da9f1fb247693c1"
              }
            ]
          }
        ]
      },
      {
        "uuid": "dfa35519-9734-4259-bba1-3e825cf4be06",
        "name": "Vulnerability Disclosure Report",
        "type": "VULNERABILITIES",
        "formats": [
          {
            "mediaType": "application/vnd.cyclonedx+xml",
            "description": "CycloneDX VDR (XML)",
            "url": "https://tomcat.apache.org/cyclonedx/vdr.xml",
            "checksums": [
              {
                "algType": "SHA-256",
                "algValue":
"75b81020b3917cb682b1a7605ade431e062f7a4c01a412f0b87543b6e995ad2a"
              }
            ]
          }
        ]
```

```
        }
      ]
    }
  }
```

7.20 date-time

Timestamp

Type: string

Format: date-time

Pattern: `^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$`

Example (Informative)

```
2024-03-20T15:30:00Z
```

7.21 discovery-info

Discovery information for a TEI

Type: object

Table 66: Properties

Property	Type	Requirement	Description
productReleaseUuid	**uuid**	Required	UUID of the resolved TEA Product Release
servers	Array of **tea-server-info**	Required	Array of TEA server information

7.22 error-response

Error response

Type: object

Table 67: Properties

Property	Type	Requirement	Description
error	**unknown-error-type**	Required	

7.23 identifier

An identifier with a specified type

Type: object

Table 68: Properties

Property	Type	Requirement	Description
idType	identifier-type	Optional	Type of identifier, e.g. **TEI**, **PURL**, **CPE**
idValue	string	Optional	Identifier value

7.24 identifier-type

Enumeration of identifiers types

Type: string

Table 69: Enumeration of possible values

Value
CPE
TEI
PURL
COMPLIANCE_DOCUMENT

7.25 paginated-component-release-response

A paginated response containing TEA Component Releases

Type: object

7.26 paginated-component-response

A paginated response containing TEA Components

Type: object

7.27 paginated-product-release-response

A paginated response containing TEA Product Releases

Type: object

7.28 paginated-product-response

A paginated response containing TEA Products

Type: object

7.29 pagination-details

Type: object

Table 70: Properties

Property	Type	Requirement	Description
timestamp	string	Required	
pageStartIndex	integer	Required	
pageSize	integer	Required	
totalResults	integer	Required	

7.30 product

A TEA product

Type: object

Table 71: Properties

Property	Type	Requirement	Description
uuid	uuid	Required	A unique identifier for the TEA product
name	string	Required	Product name
identifiers	Array of identifier	Required	List of identifiers for the product, like TEI, CPE, PURL or other identifiers

Example (Informative)

```
{
  "uuid": "09e8c73b-ac45-4475-acac-33e6a7314e6d",
  "name": "Apache Log4j 2",
  "identifiers": [
    {
      "idType": "CPE",
      "idValue": "cpe:2.3:a:apache:log4j"
    },
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.logging.log4j/log4j-api"
    }
  ]
}
```

7.31 productRelease

A specific release of a TEA product

Type: object

Table 72: Properties

Property	Type	Requirement	Description
uuid	uuid	Required	A unique identifier for the TEA Product Release

Table 72: Properties *(continued)*

Property	Type	Requirement	Description
product	uuid	Optional	UUID of the TEA Product this release belongs to
productName	string	Optional	Name of the TEA Product this release belongs to
version	string	Required	Version number of the product release
createdDate	date-time	Required	Timestamp when this Product Release was created in TEA (for sorting purposes)
releaseDate	date-time	Optional	Timestamp of the product release
preRelease	boolean	Optional	A flag indicating pre-release (or beta) status. May be disabled after the creation of the release object, but can't be enabled after creation of an object.
identifiers	Array of identifier	Optional	List of identifiers for the product release
components	Array of component-ref	Required	List of component references that compose this product release. A component reference can optionally include the UUID of a specific component release to pin the exact version.

Example (Informative)

```
{
  "uuid": "123e4567-e89b-12d3-a456-426614174000",
  "version": "2.24.3",
  "createdDate": "2025-04-01T15:43:00.000Z",
  "releaseDate": "2025-04-01T15:43:00.000Z",
  "identifiers": [
    {
      "idType": "TEI",
      "idValue": "tei:vendor:product@2.24.3"
    }
  ],
  "components": [
    {
      "uuid": "3910e0fd-aff4-48d6-b75f-8bf6b84687f0"
    },
    {
      "uuid": "b844c9bd-55d6-478c-af59-954a932b6ad3",
      "release": "da89e38e-95e7-44ca-aa7d-f3b6b34c7fab"
    }
  ]
}
```

7.32 release

A TEA Component Release

Type: object

Table 73: Properties

Property	Type	Requirement	Description
uuid	**uuid**	Required	A unique identifier for the TEA Component Release
component	**uuid**	Optional	UUID of the TEA Component this release belongs to
componentName	string	Optional	Name of the TEA Component this release belongs to
version	string	Required	Version number
createdDate	**date-time**	Required	Timestamp when this Release was created in TEA (for sorting purposes)
releaseDate	**date-time**	Optional	Timestamp of the release
preRelease	boolean	Optional	A flag indicating pre-release (or beta) status. May be disabled after the creation of the release object, but can't be enabled after creation of an object.
identifiers	Array of **identifier**	Optional	List of identifiers for the component
distributions	Array of **release-distribution**	Optional	List of different formats of this component release

Example 1 (Informative)

```
7-40e4-9abf-c400b10f0345",

05-07T18:08:00.000Z",
05-12T18:08:00.000Z",



aven/org.apache.tomcat/tomcat@11.0.7"



 "6a0d58e1-4896-4f1e-83f7-5feb6c032537",
ore binary distribution, zip archive",


RL",
kg:maven/org.apache.tomcat/tomcat@11.0.6?type=zip"



HA_256",
9da736a1cdd27231e70187cbc67398d29ca0b714f885e7032da9f1fb247693c1"


epo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6.zip",
https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6.z


 "dba01c13-dd96-4928-be72-9c87ffa8cab8",
ore binary distribution, tar.gz archive",


RL",
kg:maven/org.apache.tomcat/tomcat@11.0.6?type=tar.gz"



HA_256",
2fcece641c62ba1f28e1d7b257493151fc44f161fb391015ee6a95fa71632fb9"


epo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6.tar.gz",
https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6.t


 "cfe068c2-fae7-43d0-97ec-5ea092454040",
ore binary distribution, Windows x64 zip archive",


RL",
kg:maven/org.apache.tomcat/tomcat@11.0.6?classifier=windows-x64&type=zip"



HA_256",
62a5c358d87a8ef21d7ec1b3b63c9bbb577453dda9c00cbb522b16cee6c23fc4"
```

```
epo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6-windows-x6
https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/11.0.7/tomcat-11.0.6.z


 "de45ffaf-e4b5-47b5-be28-444a76df098e",
ore binary distribution, Windows Service Installer (MSI)",


HA_512",


bd9e67b14a60f2aaa6aa7775116bce40eb0579e8ced162a4f828051d3b867e96ee2858ec5da0cc654e83


lcdn.apache.org/tomcat/tomcat-11/v11.0.7/bin/apache-tomcat-11.0.7.exe",
https://downloads.apache.org/tomcat/tomcat-11/v11.0.7/bin/apache-tomcat-11.0.7.exe.a
```

Example 2 (Informative)

```
{
  "uuid": "95f481df-f760-47f4-b2f2-f8b76d858450",
  "version": "11.0.0-M26",
  "createdDate": "2024-09-13T17:49:00.000Z",
  "preRelease": true,
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.0-M26"
    }
  ]
}
```

7.33 release-distribution

Type: object

Table 74: Properties

Property	Type	Requirement	Description
distributionId	**uuid**	Required	A unique identifier for the TEA Distribution object
description	string	Optional	Free-text description of the distribution.
identifiers	Array of **identifier**	Optional	List of identifiers specific to this distribution.
url	string	Optional	Direct download URL for the distribution.
signatureUrl	string	Optional	Direct download URL for the distribution's external signature.
checksums	Array of **checksum**	Optional	List of checksums for the distribution.

Example 1 (Informative)


```
{
  "distributionId": "6a0d58e1-4896-4f1e-83f7-5feb6c032537",
  "description": "Core binary distribution, zip archive",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.6?type=zip"
    }
  ],
  "checksums": [
    {
      "algType": "SHA_256",

"algValue":
"9da736a1cdd27231e70187cbc67398d29ca0b714f885e7032da9f1fb247693c1"
    }
  ],
  "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/
11.0.7/tomcat-11.0.6.zip",
  "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/
tomcat/tomcat/11.0.7/tomcat-11.0.6.zip.asc"
}
```


Example 2 (Informative)


```
{
  "distributionId": "dba01c13-dd96-4928-be72-9c87ffa8cab8",
  "description": "Core binary distribution, tar.gz archive",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/tomcat@11.0.6?type=tar.gz"
    }
  ],
  "checksums": [
    {
      "algType": "SHA_256",

"algValue":
"2fcece641c62ba1f28e1d7b257493151fc44f161fb391015ee6a95fa71632fb9"
    }
  ],
  "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/
11.0.7/tomcat-11.0.6.tar.gz",
  "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/
tomcat/tomcat/11.0.7/tomcat-11.0.6.tar.gz.asc"
}
```


Example 3 (Informative)

```
{
  "distributionId": "cfe068c2-fae7-43d0-97ec-5ea092454040",
  "description": "Core binary distribution, Windows x64 zip archive",
  "identifiers": [
    {
      "idType": "PURL",
      "idValue": "pkg:maven/org.apache.tomcat/
tomcat@11.0.6?classifier=windows-x64&type=zip"
    }
  ],
  "checksums": [
    {
      "algType": "SHA_256",

"algValue":
"62a5c358d87a8ef21d7ec1b3b63c9bbb577453dda9c00cbb522b16cee6c23fc4"
    }
  ],
  "url": "https://repo.maven.apache.org/maven2/org/apache/tomcat/tomcat/
11.0.7/tomcat-11.0.6-windows-x64.zip",
  "signatureUrl": "https://repo.maven.apache.org/maven2/org/apache/
tomcat/tomcat/11.0.7/tomcat-11.0.6.zip.asc"
}
```

Example 4 (Informative)

```
45ffaf-e4b5-47b5-be28-444a76df098e",
binary distribution, Windows Service Installer (MSI)",


12",


bd9e67b14a60f2aaa6aa7775116bce40eb0579e8ced162a4f828051d3b867e96ee2858ec5da0cc654e83


.apache.org/tomcat/tomcat-11/v11.0.7/bin/apache-tomcat-11.0.7.exe",
s://downloads.apache.org/tomcat/tomcat-11/v11.0.7/bin/apache-tomcat-11.0.7.exe.asc"
```

7.34 tea-server-info

TEA server information including URL, versions, and optional priority

Type: object

Table 75: Properties

Property	Type	Requirement	Description
`rootUrl`	string	Required	Root URL of the TEA server for this TEI without trailing slash
`versions`	Array of string	Required	Supported TEA API versions at this server without v prefix
`priority`	number	Optional	Optional priority for this server (`0.0` to `1.0`, where `1.0` is highest priority)

7.35 unknown-error-type

Classification of TEA error response

Type: string

Table 76: Enumeration of possible values

Value
`OBJECT_UNKNOWN`
`OBJECT_NOT_SHAREABLE`

7.36 uuid

A UUID

Type: string

Format: uuid

Pattern: `^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$`

Annex A
(informative)

Bibliography

SKELETON — to be authored by the working group.

- CycloneDX project — https://cyclonedx.org/.
- SPDX project — https://spdx.dev/.
- OpenVEX project — https://openvex.dev/.
- OASIS CSAF — https://oasis-open.github.io/csaf-documentation/.
- NTIA Minimum Elements for an SBOM (2021).
- Executive Order 14028 — Improving the Nation's Cybersecurity.

Annex B
(informative)

Colophon

SKELETON — to be authored by the working group.

This document was produced from the Transparency Exchange API <https://github.com/CycloneDX/transparency-exchange-api> source repository using Ecmarkup <https://github.com/tc39/ecmarkup>, the standards-authoring tool maintained by Ecma TC39.

The body of the Specification is assembled from two sources of truth: the OpenAPI document at **`spec/openapi.yaml`**, from which the API surface and data model are generated; and a set of Markdown documents in the repository, which carry the narrative chapters and are imported verbatim. A small build pipeline under **`ecmarkup/`** converts both into Ecmarkup HTML and renders it via the standard Ecmarkup toolchain.

Software License

Ecma International
Rue du Rhone 114
CH-1204 Geneva
Tel: +41 22 849 6000
Fax: +41 22 849 6001
Web: https://ecma-international.org/